# CoworkingHub: Sistema de Gestión de Reservas

## Arquitectura Desacoplada, Escalabilidad y Seguridad

Luis Toscano, Julio Gutierrez  Jhon Amaya

14 de abril de 2026

# Propósito del Proyecto

- **Problema detectado:** La gestión ineficiente y manual de reservas de oficinas y escritorios compartidos.
- **Solución implementada:** Una aplicación web escalable que unifica el registro de usuarios, la autenticación segura y la gestión de reservas en una interfaz intuitiva.
- **Beneficios clave:**
  - Experiencia de usuario (UX) fluida y sin recargas innecesarias.
  - Separación de responsabilidades mediante una arquitectura cliente-servidor (Desacoplamiento).
  - Infraestructura lista para integraciones continuas y despliegue en la nube.

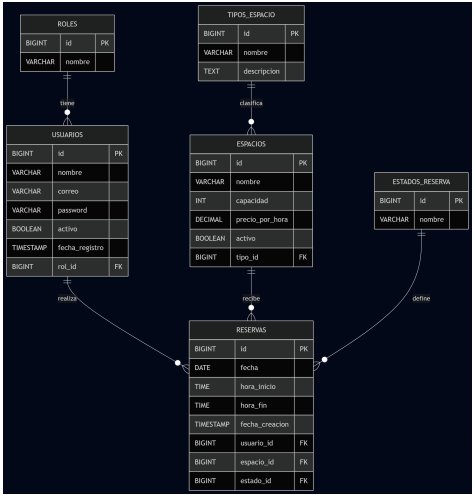
# Historias de Usuario

El desarrollo se basó en el flujo directo de las necesidades de Coworking:

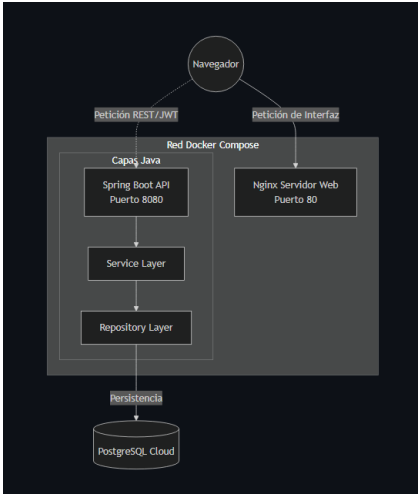
| ID | Historia de Usuario | Objetivo Principal |
|---|---|---|
| **HU01** | Registro de Usuario | Solicitar datos y prevenir duplicidad de correos. |
| **HU02** | Iniciar Sesión | Validar credenciales y emitir token seguro (JWT). |
| **HU03** | Disponibilidad | Mostrar espacios ocupados y libres según fecha. |
| **HU04** | Crear Reserva | Registrar alquiler evitando sobreposiciones. |
| **HU05** | Cancelar Reserva | Liberar espacio (Acción por Dueño o Admin). |
| **HU06** | Gestión (Admin) | Crear, editar y definir tipos y precios de espacio. |

# Diseño de Base de Datos y Arquitectura

## Arquitectura base de datos




## Arquitectura Servicios



# Tecnologías de Desarrollo

**Cambio de Paradigma:** De monolítico a arquitectura desacoplada.

**Backend (El motor)**

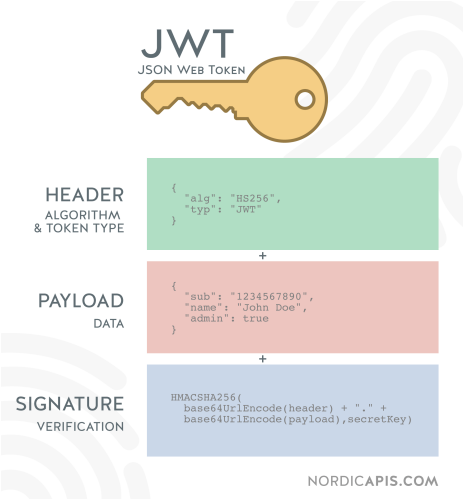
- Desarrollado en **Java 17** con **Spring Boot 3**.
- **Spring Data JPA** ligado a **PostgreSQL**.
- **Flyway** para migraciones de BD seguras.

**Frontend e Infraestructura**

- Interfaz en **Vanilla JS, HTML5 y CSS3** puro para control total.
- Uso de **Nginx** como proxy inverso moderno para manejar CORS y archivos estáticos.

# ¿Qué es JWT y por qué es importante?

- **¿Qué es?:** JSON Web Token (JWT) es un estándar abierto para la transmisión segura de información entre partes.
- **Estructura Base:**
  - *Header:* Tipo y Algoritmo.
  - *Payload:* Datos del usuario (Claims).
  - *Signature:* Firma inviolable.
- **Importancia Clave:**
  - **Stateless:** El servidor no almacena sesiones.
  - **Escalabilidad:** Permite la adición de nodos al backend de forma inmediata.



# Implementación de JWT en CoworkingHub

**Tecnología:** Todo se respalda en **Spring Security** + `jjwt`.

**El Flujo de Seguridad Paso a Paso:**

1. **Autenticación:** Usuario ingresa y la contraseña se valida con **BCrypt**.
2. **Emisión:** El backend retorna un nuevo *Access Token*.
3. **Almacenamiento:** El Frontend lo recibe y lo aloja sútilmente en el `localStorage`.
4. **Autorización Constante:** Para navegar y hacer reservas, el token es adjuntado internamente en las cabeceras HTTP:
   `Authorization: Bearer <token>`

# Containerización con Docker

- **Aislamiento de Aplicación:** Todo orquestado vía **Docker Compose**.
- **Contenedores Creados:**
  - Contenedor exclusivo para Java (Backend API).
  - Contenedor exclusivo para Nginx (Frontend Web).
- **Ventajas de este formato:**
  - Control absoluto del entorno: "Si funciona en mi local, funciona aquí".
  - Adaptación nativa y simplificada para los despliegues ágiles en servidores Cloud como **Render**.

# Consideraciones, Retos y Soluciones

- **Refactorización de UI:** Unificar *Registro* y *Login* y omitir el *scroll*, ofreciendo una vista central pura.
- **Enrutamiento de Nginx:** Resolver los fallos "Not Found". Tuvimos que reconfigurar la interceptación de `/api` en Nginx para mandarlas ocultas a Spring Boot.
- **Robustez del Frontend:** Empleo profundo de 'fetch' interceptando los errores. Creación de estados de Çarga..."(*loading states*) para darle tranquilidad al usuario cuando la DB está cargando.

# Demostración en Vivo

Tour por la Arquitectura Front/Back de CoworkingHub

# Conclusiones y Visiones a Futuro

- **Conclusión:** Culminamos una base de código limpia. CoworkingHub permite hoy escalar y soportar un volumen altísimo gracias al JWT y al desacople total entre el visual (*Nginx*) y la lógica (*Java*).

- **Trabajo a Futuro en la Plataforma:**
  - Un mapa visual e interactivo de distribución de los escritorios.
  - Integración con una pasarela real de pagos (ej: Stripe, PayPal).
  - Un Panel para que el dueño analice picos de horas muertas.